

# Documento Detallado sobre Arquitecturas de Desarrollo de Software

## Introducción

Este documento proporciona un análisis detallado de las arquitecturas de desarrollo de software más utilizadas en empresas de diversos sectores: bancario, software (general), salud, educación, comercio electrónico y streaming. Se basa en investigaciones recientes sobre tendencias en 2025-2026. Para cada sector, se incluyen definiciones, razones de uso, ejemplos de empresas y estructuras de carpetas comunes. Las arquitecturas más recurrentes son: monolítica, microservicios, serverless, en capas (n-tier) y basada en eventos.

Se utilizan tablas para organizar la información de manera clara y estructurada.

## Sector Bancario

En el sector bancario, las arquitecturas deben priorizar la seguridad, escalabilidad y cumplimiento normativo (ej. HIPAA, GDPR). Las más usadas son microservicios para flexibilidad y monolítica para sistemas legacy, con un enfoque en API y cloud.

| Arquitectura   | Definición                                                                                                      | Por qué se usa                                                                                                                                                                                 | Empresas Ejemplos                                                                             | Estructura de Carpetas Común                                                                                                                                                            |
|----------------|-----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Microservicios | Aplicación dividida en servicios independientes que se comunican vía API, cada uno con su propia base de datos. | Mejora escalabilidad, resiliencia y seguridad; permite actualizaciones independientes sin afectar el sistema completo. Ideal para manejar transacciones en tiempo real y cumplir regulaciones. | JPMorgan Chase, HSBC, Bank of America (usan microservicios para apps móviles y core banking). | app/<br>├── service1/<br>(e.g., payments)<br>    ├── src/<br>    ├── tests/<br>    └──<br>Dockerfile<br>├── service2/<br>(e.g., authentication)<br>└── shared/<br>(bibliotecas comunes) |

|                   |                                                                                                          |                                                                                                           |                                                                                      |                                                                                                                        |
|-------------------|----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Monolítica        | Aplicación como una unidad única con codebase centralizado.                                              | Simplicidad en desarrollo inicial y mantenimiento de legacy; bajo costo para sistemas estables.           | Wells Fargo (sistemas core legacy), BBVA (transición de monolitos a microservicios). | app/<br>├──<br>controllers/<br>├── models/<br>├── views/<br>├── config/<br>└── database/                               |
| Serverless        | Código ejecutado en funciones gestionadas por el proveedor (e.g., AWS Lambda), sin servidores dedicados. | Reduce costos operativos y escala automáticamente; útil para procesamiento de eventos como transacciones. | Capital One, Nubank.                                                                 | functions/<br>├── lambda1.js (e.g., auth)<br>├── lambda2.js (e.g., transactions)<br>└── serverless.yml (configuración) |
| En Capas (n-tier) | Dividida en capas: presentación, lógica de negocio, datos.                                               | Facilita separación de preocupaciones y escalabilidad por capa; común en apps bancarias para seguridad.   | Citibank, Santander.                                                                 | app/<br>├── presentation/ (UI)<br>├── business/ (lógica)<br>├── data/ (acceso a DB)<br>└── infrastructure/             |

## Sector Software (General)

En la industria de software general, se prioriza la flexibilidad y escalabilidad. Microservicios dominan para apps grandes, mientras que monolítica se usa en startups.

| Arquitectura | Definición | Por qué se usa | Empresas Ejemplos | Estructura de Carpetas Común |
|--------------|------------|----------------|-------------------|------------------------------|
|              |            |                |                   |                              |

|                   |                                                    |                                                                                       |                                         |                                                                                                                                                                              |
|-------------------|----------------------------------------------------|---------------------------------------------------------------------------------------|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Microservicios    | Servicios independientes que forman una app mayor. | Permite desarrollo paralelo, escalabilidad y resiliencia; ideal para equipos grandes. | Google, Amazon, Microsoft (Azure apps). | <pre>project/ ├── ├── microservice1/ │   ├── api/ │   ├── domain/ │   └── ├── infrastructure/ │   ├── │   ├── microservice2/ │   │   └── api- │   │       └── gateway/</pre> |
| Monolítica        | Unidad única con todo integrado.                   | Fácil para prototipos y apps pequeñas; menor complejidad inicial.                     | Basecamp, early versions of Shopify.    | <pre>app/ ├── src/ ├── tests/ ├── config/ └── docs/</pre>                                                                                                                    |
| Basada en Eventos | Componentes reaccionan a eventos (e.g., Kafka).    | Maneja datos en tiempo real y desacoplamiento; útil para apps distribuidas.           | Uber, LinkedIn.                         | <pre>app/ ├── events/ ├── handlers/ ├── streams/ └── producers/</pre>                                                                                                        |
| En Capas          | Capas separadas por función.                       | Organiza código lógicamente; facilita mantenimiento.                                  | Oracle, SAP.                            | <pre>app/ ├── ui/ ├── services/ ├── repository/ └── entities/</pre>                                                                                                          |

## Sector Salud

En salud, se enfatiza la interoperabilidad (e.g., FHIR) y seguridad. Microservicios para apps modernas, monolítica en EHR legacy.

| Arquitectura | Definición | Por qué se usa | Empresas Ejemplos | Estructura de Carpetas Común |
|--------------|------------|----------------|-------------------|------------------------------|
|              |            |                |                   |                              |

|                |                                                             |                                                                                         |                                                     |                                                                                                                                           |
|----------------|-------------------------------------------------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Microservicios | Servicios autónomos para funciones como EHR o telemedicina. | Escalabilidad para datos masivos; integración con dispositivos IoT; cumplimiento HIPAA. | Epic Systems, Cerner (transición a microservicios). | <pre> health-app/ ├── patient-service/ │   ├── controllers/ │   │   └── models/ │   └── billing-service/ │       └── auth-service/ </pre> |
| Monolítica     | App integrada para EHR.                                     | Simplicidad en sistemas legacy; control centralizado.                                   | Allscripts, McKesson.                               | <pre> app/ ├── modules/ │   (e.g., patient, admin) ├── core/ └── database/ </pre>                                                         |
| Serverless     | Funciones para procesamiento de datos.                      | Bajo costo para apps con picos (e.g., telemedicina); escalabilidad automática.          | Philips Healthcare, GE Healthcare.                  | <pre> functions/ ├── process-data.js ├── notify.js └── serverless-config.yml </pre>                                                       |
| En Capas       | Capas para UI, lógica y datos.                              | Separación para seguridad y compliance.                                                 | Siemens Healthineers.                               | <pre> app/ ├── presentation/ ├── application/ │   ├── domain/ │   └── └── infrastructure/ </pre>                                          |

## Sector Educación

En educación, se usa para LMS y apps de e-learning. Microservicios para escalabilidad en plataformas grandes.

| Arquitectura | Definición | Por qué se usa | Empresas Ejemplos | Estructura de Carpetas Común |
|--------------|------------|----------------|-------------------|------------------------------|
|              |            |                |                   |                              |

|                   |                                        |                                                                                  |                                                     |                                                                                                                              |
|-------------------|----------------------------------------|----------------------------------------------------------------------------------|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Microservicios    | Servicios para cursos, usuarios, etc.  | Escalabilidad para usuarios concurrentes; integración con herramientas externas. | Coursera, edX (usando microservicios para escalar). | edu-<br>platform/<br>├─ course-<br>service/<br>  ├─ api/<br>  └─ db/<br>├─ user-<br>service/<br>└─<br>analytics-<br>service/ |
| Monolítica        | App única para LMS.                    | Fácil desarrollo para plataformas pequeñas.                                      | Moodle (base monolítica), Blackboard.               | app/<br>├─ core/<br>├─ plugins/<br>├─ themes/<br>└─ lang/                                                                    |
| En Capas          | Capas para UI y backend.               | Organización clara para mantenimiento.                                           | Canvas LMS.                                         | app/<br>├─<br>frontend/<br>├─<br>backend/<br>├─<br>database/<br>└─ utils/                                                    |
| Basada en Eventos | Reacción a eventos como inscripciones. | Personalización en tiempo real.                                                  | Duolingo.                                           | app/<br>├─ event-<br>bus/<br>├─<br>listeners/<br>└─<br>emitters/                                                             |

Sector Comercio Electrónico

En e-commerce, escalabilidad es clave. Microservicios para manejar tráfico variable.

| Arquitectura | Definición | Por qué se usa | Empresas Ejemplos | Estructura de Carpetas Común |
|--------------|------------|----------------|-------------------|------------------------------|
|              |            |                |                   |                              |

|                |                                      |                                                            |                                                      |                                                                                                                                                    |
|----------------|--------------------------------------|------------------------------------------------------------|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Microservicios | Servicios para catálogo, pagos, etc. | Escalabilidad en picos (e.g., Black Friday); flexibilidad. | Amazon, eBay, Shopify (transición a microservicios). | <pre> ecommerce/ ├── catalog- │   └── service/ │       ├── src/ │       └── tests/ ├── cart- │   └── service/ └── payment-     └── service/ </pre> |
| Monolítica     | App integrada.                       | Simplicidad para tiendas pequeñas.                         | Early WooCommerce sites.                             | <pre> app/ ├── public/ ├── includes/ ├── admin/ └── wp-     └── content/ </pre>                                                                    |
| Headless       | Backend separado de frontend.        | Flexibilidad en UI.                                        | BigCommerce, Magento.                                | <pre> app/ ├── api/ ├── frontend/ └── backend/ </pre>                                                                                              |
| Serverless     | Funciones para checkout.             | Bajo costo en tráfico variable.                            | Zalando.                                             | <pre> functions/ ├── checkout.js ├── inventory.js └── config.yml </pre>                                                                            |

Sector Streaming

En streaming, se necesita baja latencia y escalabilidad masiva. Microservicios y basada en eventos son dominantes.

| Arquitectura | Definición | Por qué se usa | Empresas Ejemplos | Estructura de Carpetas Común |
|--------------|------------|----------------|-------------------|------------------------------|
|              |            |                |                   |                              |

|                   |                                          |                                                       |                             |                                                                                                                                                                   |
|-------------------|------------------------------------------|-------------------------------------------------------|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Microservicios    | Servicios para video, recomendaciones.   | Escalabilidad global; actualizaciones independientes. | Netflix, Spotify.           | <pre> streaming-app/ ├── video- │   └── service/ │       ├── encoders/ │       └── storage/ └── recommendation-     └── service/         └── user-service/ </pre> |
| Basada en Eventos | Procesamiento de streams en tiempo real. | Manejo de datos en vivo (e.g., Kafka).                | YouTube, Hulu.              | <pre> app/ ├── streams/ ├── processors/ └── sinks/ </pre>                                                                                                         |
| Serverless        | Funciones para transcoding.              | Escalabilidad para picos de viewers.                  | Twitch (partes serverless). | <pre> functions/ ├── transcode- │   └── video.js ├── notify-user.js └── serverless.yml </pre>                                                                     |
| En Capas          | Capas para entrega de contenido.         | Optimización de rendimiento.                          | Disney+.                    | <pre> app/ ├── delivery- │   └── layer/ ├── processing- │   └── layer/ └── storage-layer/ </pre>                                                                  |

## Conclusión

Las arquitecturas evolucionan hacia microservicios y serverless para mayor escalabilidad, aunque monolítica persiste en legacy. La elección depende de tamaño, tráfico y necesidades de compliance. Estructuras de carpetas promueven modularidad y mantenimiento. Para más detalles, consulte fuentes como AWS o Microsoft Azure.